

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук

Образовательная программа «Дизайн и разработка информационных продуктов»

СОГЛАСОВАНО

Научный руководитель
Приглашённый преподаватель:
Факультет компьютерных наук /
Департамент больших данных и
информационного поиска / Базовая
кафедра Т-Банка

_____ А. А. Шкель
«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
«Дизайн и разработка
информационных продуктов»,
преподаватель базовой кафедры Т-
Банка

_____ Д. Д. Син
«__» _____ 2026 г.

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Техническое задание (индивидуальная часть)

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.03-04 ТЗ 03-1-ЛУ

Исполнители:

Студент группы БДРИП242

_____ / А. А. Клушин /

«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл	

УТВЕРЖДЕН

RU.17701729.09.03-04 ТЗ 03-1-ЛЮ

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Техническое задание (индивидуальная часть)

RU.17701729.09.03-04 ТЗ 03-1

Листов 12

Москва 2026

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Настоящий документ является индивидуальной частью технического задания на разработку программы «Платформа для no-code автоматизации бизнес-процессов» в рамках командной курсовой работы (общая часть — RU.17701729.09.03-04 ТЗ 01-1). Документ определяет зону ответственности студента группы БДРИП242 Клушина А.А. в составе проектной группы из двух человек.

Индивидуальная часть распространяется на серверную часть программы (Kotlin + Spring Boot), движок исполнения workflow, исполнители узлов, песочницу пользовательского Python-кода, инфраструктуру (PostgreSQL, MinIO, Liquibase, Docker, CI/CD) и автоматизированные тесты серверной части.

Настоящий документ разработан в соответствии с ГОСТ 19.201-78.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	4
1.1. Связь с общей частью технического задания	4
1.2. Состав команды и распределение зон ответственности	4
2. ИНДИВИДУАЛЬНАЯ ЗОНА ОТВЕТСТВЕННОСТИ	5
2.1. Перечень индивидуальных функциональных требований	5
2.2. Требования к инфраструктуре и сборке	7
2.3. Требования к документации REST API	8
2.4. Требования к автоматическому тестированию	8
2.5. Используемые технологии	8
3. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	11

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Связь с общей частью технического задания

Настоящий документ дополняет общую часть технического задания на разработку программы «Платформа для no-code автоматизации бизнес-процессов» (шифр RU.17701729.09.03-04 ТЗ 01-1), уточняя обязательства студента группы БДРИП242 Клушина Артёма Алексеевича в части, относящейся к серверному слою и инфраструктуре системы. В случае несоответствия между настоящим документом и общей частью приоритет имеет общая часть. Настоящий документ разработан в соответствии с ГОСТ 19.201-78.

1.2. Состав команды и распределение зон ответственности

Командный курсовой проект «Платформа для no-code автоматизации бизнес-процессов» выполняется проектной группой из двух участников:

— **Клушин Артём Алексеевич** (группа БДРИП242) — ответственный за серверную часть программы (Kotlin + Spring Boot), инфраструктуру (PostgreSQL, MinIO, Docker, Liquibase), автоматизацию тестирования (JUnit 5, Testcontainers), пайплайн непрерывной интеграции и развёртывания (GitHub Actions, Docker Compose);

— **Гаврилин Станислав Васильевич** (группа БДРИП241) — ответственный за клиентскую часть программы (Angular, TypeScript), пользовательский опыт, end-to-end-тестирование на Playwright (RU.17701729.09.03-04 ТЗ 02-1).

Научный руководитель проекта — Шкель Алексей Алексеевич, базовая кафедра Т-Банка факультета компьютерных наук НИУ ВШЭ.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. ИНДИВИДУАЛЬНАЯ ЗОНА ОТВЕТСТВЕННОСТИ

2.1. Перечень индивидуальных функциональных требований

В рамках индивидуальной части студент Клушин А.А. обеспечивает выполнение следующих функциональных требований общей части (нумерация требований соответствует разделу 4.1.1 общей части ТЗ):

1) **Требование 2 — система триггеров.** Реализация трёх типов триггеров: ручной запуск через REST-эндпойнт `POST /v1/workflows/{id}/runs`; scheduler-триггер с поддержкой cron-выражений через `Spring TaskScheduler` и горячей перезагрузкой расписания при изменении конфигурации; `webhook`-триггер со сгенерированным уникальным токеном и приёмом произвольного JSON-тела через `POST /v1/webhook/{token}` в качестве входа `workflow`.

2) **Требование 3 — HTTP-узел.** Реализация исполнителя `HttpNodeExecutor` с поддержкой HTTP-методов `GET`, `POST`, `PUT`, `PATCH`, `DELETE`; настраиваемых заголовков; тела запроса в формате `JSON`; настраиваемого тайм-аута запроса (по умолчанию 30 секунд, максимум 300); корректной обработки ошибочных ситуаций (`DNS`-ошибки, `connection-refused`, тайм-ауты, статусы `4xx/5xx`) с фиксацией диагностических сообщений в `node_run.error_message`.

3) **Требование 4 — узлы потоков данных.** Реализация пяти исполнителей потоковых операций (`filter` с булевым предикатом, `map` с трансформирующим выражением, `reduce` с агрегирующим выражением и начальным значением, `foreach` со `side`-эффектным выражением, `flatMap` с разворачивающим выражением) в классе `DataflowNodeExecutors`. Поддержка проверки типов на вход (только списки), корректной обработки пустых списков, передачи контекста выражения через переменные `item` и `acc`.

4) **Требование 5 — Code Node с изолированной песочницей.** Реализация исполнителя `PythonNodeExecutor`, запускающего пользовательский Python-код в одноразовых `Docker`-контейнерах. Полная изоляция: `--network none`, `--read-only`, `--tmpfs /tmp:rw,size=16m`, `--cap-drop ALL`, `--security-opt no-new-privileges`, `--memory 256m`, `--cpus 1`, `--pids-limit 64`. Тайм-аут исполнения 5 секунд (с `overhead` 30 секунд на запуск контейнера). Базовый образ — `python:3.11-slim`. Поддержка `JavaScript` реализована классом `JavaScriptNodeExecutor` через тот же универсальный движок `ContainerSandboxRunner` с базовым образом `node:20-alpine`.

5) **Требование 6 — параллельное исполнение workflow.** Реализация `WorkflowExecutionService` с топологической сортировкой графа по алгоритму Кана (с ранней детекцией циклов), построением цепочки `CompletableFuture.allOf(...).thenApplyAsync(...)`

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

в пуле потоков workflowExecutor размера 16, изоляцией сбоев одной ветви от параллельных, транзитивным распространением статуса skipped по потомкам сбойного узла.

6) **Требование 7 — версионирование и snapshot-механизм.** Реализация двухуровневой схемы хранения: автоматических ревизий (таблица workflow_revision, инкрементальный revision_number, создание при каждом сохранении) и именованных версий-снапшотов (таблица workflow_version, FK root_revision_id с ON DELETE RESTRICT). Реализация REST-эндпойнтов управления версиями (GET /v1/workflows/{id}/versions, POST /v1/workflows/{id}/versions, POST /v1/workflows/{id}/versions/{vid}/restore). Восстановление к версии реализуется через создание новой ревизии с прошлым graph_skeleton (append-only-семантика, иммутабельность исторических данных).

7) **Требование 10 — WebSocket-трансляция событий (серверная сторона).** Реализация STOMP-эндпойнта /v1/ws с SockJS-fallback; конфигурация simple in-memory message broker с топиками /topic/...; класс GraphBroadcastListener, преобразующий доменные события (workflow_started, node_reached, node_action, node_exited, workflow_finished) в STOMP-сообщения и публикующий их в топик /topic/workflows/{workflowId}/graph через SimpMessagingTemplate.

8) **Требование 11 — контент-адресуемое хранение в MinIO.** Реализация сервиса BlobService для сохранения и чтения JSON-конфигов узлов в MinIO по схеме content-addressed; утилиты CanonicalJson (приведение JSON к канонической форме с упорядоченными ключами и нормализованным пространством) и Hashing (вычисление SHA-256 hex). Реализация таблицы blob_index (PK = content_hash) для индекса существующих хешей и быстрой проверки наличия. Идемпотентность операции put (повторная запись существующего объекта не создаёт дубликата).

9) **Требование 12 — A/B-аналитика workflow и feature-флаги (см. также требование 9 общей ТЗ).** Реализация специализированного узла BranchSplitNodeExecutor с двумя режимами работы — split (разделение входного потока на параллельные ветви для A/B-эксперимента) и pick (выбор одной ветви, применяется для feature-флагов и управляемой раскатки функциональности) — и набором стратегий выбора варианта в классе BranchSplitStrategies: random (случайное по весам), hash (детерминированное по хешу пользовательского идентификатора), modulo (по остатку от деления), attribute (по правилам атрибутов входного объекта), percentage (процентная раскатка), stratified (стратифицированное разделение). Реализация обёртки SplitEnvelope (фиксирующая выбранный вариант, оригинальный вход и

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

метаданные эксперимента) и парного узла BranchMergeNodeExecutor (слияние результатов ветвей по выбранному варианту). Реализация серверного модуля продуктовой аналитики в пакете analytics: AbAnalyticsService (агрегация просмотров и конверсий по вариантам с учётом исключённых записей), AbAnalyticsRepository (чтение variant-rows и excluded count из БД с @Transactional(readOnly = true) для согласованного чтения), StatTest (статистическая проверка значимости разности конверсий — z-тест двух пропорций с расчётом доверительных интервалов), DTO AbAnalyticsDtos. Реализация REST-эндпойнта GET /v1/workflows/{workflowId}/ab-analytics (контроллер AbAnalyticsController).

10) **Требование 13 — снимоты workflow.** Реализация механизма именованных снимотов уровня документа: таблица workflow_snapshot (миграция 005-workflow-snapshots.yaml), репозиторий WorkflowSnapshotRepository, REST-контроллер WorkflowSnapshotsController с эндпойнтами создания, чтения и удаления снимотов. Снимот фиксирует ссылку на конкретную ревизию workflow с возможностью добавить пользовательские имя и описание.

11) **Требование 14 — режим пошаговой отладки workflow.** Реализация механизма запуска одной выбранной ноды workflow с произвольным входным значением без полного прогона графа. Серверная сторона использует общий реестр исполнителей узлов и общий механизм трансляции событий жизненного цикла исполнения. Результат отладочного запуска возвращается клиентской части в синхронном виде; параллельно создаётся запись запуска с пометкой отладочного исполнения, что обеспечивает попадание результата в общую историю запусков workflow.

2.2. Требования к инфраструктуре и сборке

Серверная часть собирается утилитой Apache Maven (backend/pom.xml) с использованием JDK 24 и Spring Boot Maven Plugin для упаковки в исполняемый fat-JAR. Сборка покрытия (JaCoCo) интегрирована в фазу verify. Схема PostgreSQL 16 управляется миграциями Liquibase в формате YAML (backend/src/main/resources/db/changelog/). Локальное development-развёртывание выполняется через Docker Compose (deploy/docker-compose.dev.yml), поднимающий PostgreSQL 16 и MinIO; production-развёртывание — через deploy/docker-compose.prod.yml, дополнительно включающий контейнеры backend, frontend (Nginx с SSL), docker-socket-proxy. Непрерывная интеграция реализована через GitHub Actions (.github/workflows/ci.yml): на каждый pull request прогоняются модульные и интеграционные тесты, на каждый push в main — дополнительно сборка и публикация Docker-образов в ghcr.io, обновление SVG-бейджа покрытия в .github/badges/jacoco.svg.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2.3. Требования к документации REST API

REST API серверной части документируется через springdoc-openapi 2.7.0, публикующий спецификацию OpenAPI 3.0 по адресам `http://localhost:8080/v1/v3/api-docs` (JSON) и `/v1/v3/api-docs.yaml` (YAML), а также интерактивный интерфейс Swagger UI по адресу `http://localhost:8080/v1/swagger-ui.html`. Каждый REST-контроллер аннотирован `@Tag(name = "...")` для группировки в Swagger UI; каждый эндпоинт — `@Operation(summary = "...", description = "...")`; каждое DTO — `@Schema(description = "...")` с описанием полей. Спецификация автоматически синхронизирована с реализацией; разрыв документации и кода исключён по конструкции.

2.4. Требования к автоматическому тестированию

К моменту защиты курсовой работы серверная часть должна удовлетворять следующим целевым показателям тестового покрытия:

- модульные тесты на JUnit 5 — около 55 (с покрытием ключевых модулей: `WorkflowExecutionService`, `HttpNodeExecutor`, `PythonNodeExecutor`, `DataflowNodeExecutors`, `BlobService`, `CanonicalJson`, `Hashing`, `TriggerScheduler`, `WorkflowService`);
- интеграционные тесты на JUnit 5 + Spring Boot Test + Testcontainers — около 12 (с поднятием реальных PostgreSQL 16 и MinIO в Docker-контейнерах; покрытие сквозных сценариев: создание workflow → сохранение → запуск → проверка результатов → восстановление к версии → webhook-триггер);
- общее покрытие кода по JaCoCo (строки) — не менее 90 %; покрытие критических пакетов (`run`, `executor`, `blob`) — не менее 95 %;
- все автоматические тесты должны проходить со 100%-успехом в pipeline CI;
- время полного прогона backend-тестов — не более 8 минут на референсном раннере GitHub Actions.

2.5. Используемые технологии

Окончательный технологический стек серверной части:

- **Язык и runtime:** Kotlin 2.2.21, JDK 24, Apache Maven 3.9+;
- **Каркас приложения:** Spring Boot 3.5.3 (модули `Web`, `WebSocket`, `JDBC`, `Validation`, `Scheduler`, `Boot`);
- **Документация API:** springdoc-openapi 2.7.0;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

RU.17701729.09.03-04 ТЗ 03-1

- **Миграции БД:** Liquibase (YAML changeset);
- **Реляционная БД:** PostgreSQL 16;
- **Объектное хранилище:** MinIO (через AWS SDK for Java v2);
- **Сериализация:** Jackson Kotlin module;
- **Песочница пользовательского кода:** Docker Engine 20.10+, базовый образ `python:3.11-slim`;
- **Тестирование:** JUnit 5, Testcontainers 1.21.1, JaCoCo 0.8.13;
- **CI/CD:** GitHub Actions, Docker Compose, GitHub Container Registry (`ghcr.io`).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ

Приёмка осуществляется в соответствии с программой и методикой испытаний (индивидуальная часть, шифр RU.17701729.09.03-04 51 03-1) и подразумевает: успешное прохождение всех автоматических тестов серверной части (`mvn verify`); успешное прохождение ручных методик испытаний по каждому из одиннадцати индивидуальных функциональных требований (раздел 5 индивидуальной ПМИ); подтверждение целевых нефункциональных показателей (производительность REST API, целевое покрытие тестами, безопасность Docker-песочницы); полнота автоматической документации REST API через Swagger UI; работоспособное развёртывание через `docker compose -f deploy/docker-compose.prod.yml up`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Spring Boot Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/> (дата обращения: 05.12.2025).
2. Kotlin Language Documentation [Электронный ресурс]. — URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 08.12.2025).
3. Docker Engine Documentation [Электронный ресурс]. — URL: <https://docs.docker.com/engine/> (дата обращения: 18.03.2026).
4. MinIO Object Storage Documentation [Электронный ресурс]. — URL: <https://min.io/docs/minio/linux/index.html> (дата обращения: 20.02.2026).
5. Liquibase Documentation [Электронный ресурс]. — URL: <https://docs.liquibase.com/> (дата обращения: 15.12.2025).
6. springdoc-openapi Documentation [Электронный ресурс]. — URL: <https://springdoc.org/> (дата обращения: 05.02.2026).
7. JUnit 5 User Guide [Электронный ресурс]. — URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 08.04.2026).
8. Testcontainers for Java [Электронный ресурс]. — URL: <https://java.testcontainers.org/> (дата обращения: 12.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]